RouteSweet Development Handbook

Architecture usage troubleshooting and continuing development

OchsSoft | 26 October 2026

This handbook explains the project from everyday use to the code and data that implement it. Use it to set up a development checkout, follow a request or document through the system, locate the right extension point, make a change, and verify the result.

Begin with the project overview and architecture. The later chapters provide exact local commands, user journeys, worked development examples, diagnostics, recovery steps, known gaps, and a pinned source reference. Technical terms are explained where they are used.

The documented source is COchs-Personal/RouteSweet at b5ef1dc889ba471878ddf22db3683d33614b1858. Source-defined behavior, observed evidence, planned capabilities, and remaining platform work are distinguished. The quick operations guide remains a separate short reference.

Contents

Understand the project

Understand RouteSweet 3
Architecture and information flow 4

Use and develop the project

Install and prepare development 8
Use routes profiles and navigation 10
Continue developing the add on 13

Diagnose improve and verify

Troubleshooting known gaps and recovery 15
Source reference 17

Understand RouteSweet

RouteSweet is a Lua add-on running inside a selected retail World of Warcraft client. It gathers available activities, estimates travel and activity time, ranks a useful route, and presents navigation stops. It does not run as a Node service, a Python simulation or a standalone website. The game client owns event delivery, map and quest APIs, frames, timers and the writing of SavedVariables to disk. That runtime boundary explains most installation and troubleshooting decisions.

This repository and the installed add on

COchs-Personal/RouteSweet stores its installable source at the repository root. Install that one selected source tree as an add-on folder named RouteSweet. A second repository with a similar name is not a second component to start; it is another source revision for the same add-on identity. Installing competing copies can make you test the wrong code or overwrite saved settings. Record the exact repository, commit and game-client installation used for a test.

The inspected manifest declares Interface 120001 and a version label v0.0.3, while the bootstrap's `RS.VERSION` is 1.0.0. These are source declarations, not a fresh claim of compatibility with today's client patch. A future release should reconcile version metadata and verify the installed game API. The README's early roadmap is also behind parts of the current implementation: profiles, settings UI, leveling activity support and personal timing exist in source even though some old descriptions still list related features as future work.

The useful mental model

An activity is something the player may choose to do: a world quest, weekly task, event, dungeon or supported leveling objective. It has an ID, map coordinates, type, estimated duration, rewards, priority and possibly a quest ID or expiry. The scanner turns game API responses and expansion data into these normalized records. Normalized means different inputs are translated into the same shape before routing.

The router chooses an ordering, not a guarantee of mathematical perfection. Its nearest-neighbor pass chooses a cheap next stop; a 2-opt pass reverses segments when doing so improves the cost. Scoring and zone preferences bias that cost toward useful stops. Portal batching and navigation annotations then adapt the result to the game's geography. The displayed time is an estimate built from travel plus activity duration, and personal measurements can replace category defaults over time.

Definitions for a self taught maintainer

A table is Lua's general-purpose collection and record type. A namespace such as `RS.Scanner` groups functions under the shared add-on object. A colon call such as `RS.Scanner:GetActiveActivities()` passes that object as `self`. A callback is a function the game calls later after an event or timer. A debounce cancels a pending timer and schedules a later one, so a burst of quest changes does not trigger repeated full scans immediately.

A cost matrix stores the estimated cost from every activity to every other activity. A directed cost can differ in opposite directions because portals, hearths and cooldowns are not symmetric. A heuristic is a practical approximation used when a globally optimal solution would be expensive or unnecessary. A SavedVariable is a Lua global the client persists through the add-on manifest; it is not a database server or a file the add-on writes through ordinary operating-system file access.

What a route does and does not prove

A row appearing in the route proves that the scanner produced a candidate under current settings and that the router included it. It does not prove the objective is available to every character, that an estimated rotation is correct, that a reward is unlocked or that arriving completes the activity. Game APIs can be delayed, unavailable or restricted, and static IDs can become stale after patches. Manual marking exists for activities whose completion cannot be inferred reliably.

Treat the source-defined travel model, quest IDs and event schedules as implementation assumptions that need client evidence. This handbook explains how to inspect and repair those assumptions without inventing authoritative current game facts. Native NixOS game-client and add-on behavior require their own real qualification; reading platform-neutral Lua does not establish that the surrounding game launcher and graphics environment work.

Architecture and information flow

Load order is a dependency contract

The TOC manifest is the program's load list. It first loads the bootstrap, then the expansion registry, then expansion zone/database/activity modules, then shared consumers, and finally the UI. A Lua file existing in the folder is not enough: if it is missing from the TOC it will not run. A file loaded before the namespace or data it requires can fail before the add-on reaches initialization.

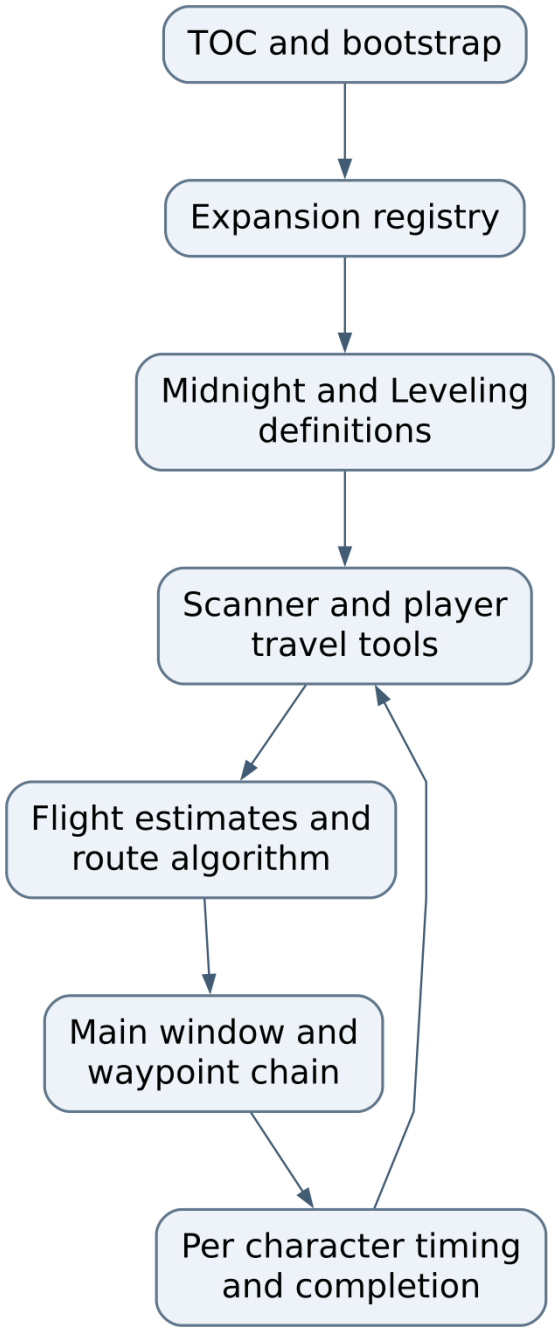


Figure 1 Load order is a dependency contract

`RS.Expansion:Register` stores an expansion's zones, level range, database and activity producers. `DetectActive` selects modules matching the character level, sorts their names deterministically and aliases `RS.DB` to a primary active database while carrying shared functions across. Zone-accessor facades preserve older call sites. This allows new content modules without duplicating the entire scanner, router or UI, but also means the primary alias and aggregated data must be checked when multiple modules match.

Scanner and scoring contract

`RS.Scanner:GetActiveActivities` combines enabled world-quest scanning with static and dynamic activities from active expansion modules. `ScoreActivity` considers activity order, reward order, expiry

and weekly-reset urgency. Settings profiles contain ordered lists of enabled activity and reward categories. Earlier enabled list entries receive stronger bonuses; disabling a category affects inclusion or weighting according to the scanner path.

World-quest data can come from task-quest and quest-log map APIs. Results are deduplicated by quest ID and filtered for completion, exclusion and near expiry. The source skips activities with less than twenty minutes remaining and marks activities under four hours as urgent. Those thresholds are implementation choices. Preserve the units in seconds when editing them; confusing API seconds with minutes produces errors that can look like a broken route optimizer.

The scanner publishes activities on `RS.activeActivities`; the wrapper builds `RS.currentRoute`. UI and navigation should consume that normalized result rather than independently asking the game for a different set of quests. A route stop normally wraps its activity in an `activity` field and adds travel, activity, arrival and departure seconds, urgency and portal notes. Synthetic portal stops can be inserted as navigation aids with zero activity duration.

Routing costs and display estimates

`BuildCostMatrix` returns two matrices. The routing matrix includes preference bias; the display matrix preserves unmodified travel estimates. Preferred destination zones subtract 180 seconds down to a minimum cost of one; avoided destinations add 180. Nearest-neighbor also subtracts a score-derived credit, with a stronger cross-zone effect and an urgent-stop boost. These artificial credits should change ordering, not make the UI claim a physically shorter flight than the model predicts.

The 2-opt improvement is capped at fifty iterations. It evaluates route edges and reverses sections when estimated cost improves. Portal-zone batching follows. Because travel can be directed and the route is then annotated with actual leg calculations, future optimizer changes need tests that measure the final route, not only one local edge-swap formula. Do not promise a globally shortest route based on the name of a heuristic.

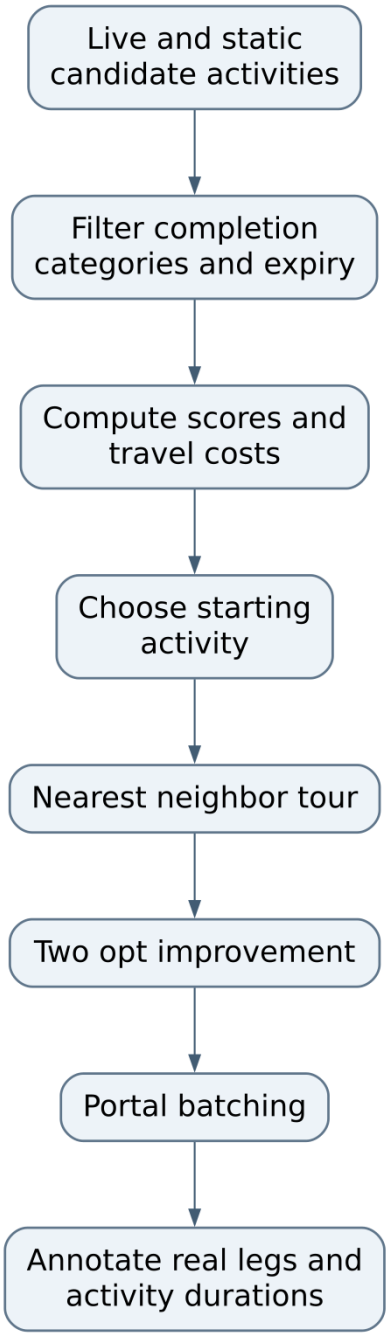


Figure 2 part 1 of 2 Routing costs and display estimates

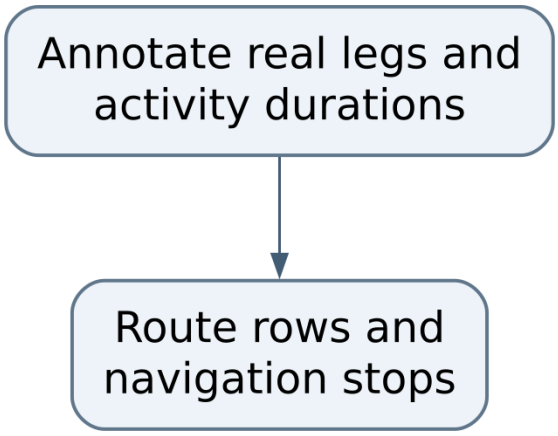


Figure 2 part 2 of 2 Routing costs and display estimates

Persistence and timing ownership

`RS_Settings` is account-level configuration. The TOC separately declares `RS_CharData` and `RS_DumpData` per character. `RS_CharData` contains manual completion, reset epoch and `activityTimes`. A timing record is `{ n, total, last }`, keyed by numeric world-quest ID or a stable string activity ID. `GetDuration` uses the rounded-up personal average first, a category default next, then `questTime *60` as fallback. Category defaults include five minutes for a world quest, twelve for a weekly, twenty for a weekly event and twenty-five for a dungeon.

`Timing:Record` ignores samples below fifteen seconds or above ninety minutes to avoid accidental clicks and AFK/disconnection outliers. Weekly reset clears manual weeklies and a choice cache, but intentionally retains personal timing. Reset detection uses the game's seconds-until-weekly-reset API and a five-minute skew allowance instead of a hardcoded regional maintenance hour. Preserve that distinction when adding migrations or data-clearing controls.

The bootstrap migrates older split world-quest timing into the unified table. Migration is code that changes old saved data into the current layout. Test it with copies of historical small fixtures before releasing a setting/schema change. Never use a real character's only SavedVariables file as the first migration test.

Install and prepare development

Install exactly the add on tree

Inspect the repository and TOC before copying files. For this repository, the repository root contains `RouteSweet.lua`, `RouteSweet.toc`, `Core`, `UI`, `Expansions` and `Textures`. The installed structure must put `RouteSweet.toc` directly inside the selected client's `Interface/AddOns/RouteSweet` directory. An extra nested download folder can make the client fail to discover the add-on. Do not copy the whole repository's documentation or version-control directory into the game by accident.

Use the actual retail game installation selected in the launcher. Different installations or compatibility prefixes can have different add-on and saved-data trees. On NixOS, the game itself is an external platform dependency; this repository does not define a pinned native launcher or a full game installation recipe. Keep that uncertainty separate from Lua source portability. Installing or changing the host's launcher/graphics configuration is outside an ordinary add-on code edit.

Enable RouteSweet in the game's AddOns list and inspect its displayed version and compatibility warning. TomTom is optional according to the source README; the current waypoint module provides native map pins. Do not add an unneeded background web server or model service to solve an add-on load failure.

Establish a safe development instance

Before updating an installed copy, exit the game normally and back up its add-on source and the relevant account and character SavedVariables files. Use a separate test character/profile where possible. The client writes SavedVariables during its own lifecycle, so editing those files while the client is running can be overwritten on exit. Keep account identifiers, character information, bag/quest data and full diagnostic dumps private.

Record the tested source commit and installed destination identity in a local report without publishing machine paths or account names. A source checkout and an installed add-on are two copies. A test result is useful only when you know which one the game loaded. Compare a deliberate harmless version or diagnostic marker when necessary, then remove temporary product clutter before publication.

Configuration in source and SavedVariables

Setting or record	Meaning	Current default or behavior
useSkyriding	Preferred flight model when live detection cannot decide	true
questTime	Fallback activity minutes	7; personal/category timing can take precedence
autoScan	Scan from relevant map events	true
waypointOnSelect	Selection navigation preference	true in defaults
portalBuffer	Additional portal transition allowance	2 minutes
combatHide	Window behavior during combat	hide_show_on_exit
activeProfile and profiles	Read-only default or custom ordered preferences	Default plus named custom profiles
completedWeeklies	Per-character manual completion markers	Cleared by real reset epoch or explicit reset
activityTimes	Per-character sample count total and last duration	Retained across normal weekly reset
RS_DumpData	Diagnostic capture	Character-scoped and potentially private

`GetActiveProfile` returns the built-in default when the requested profile is missing. `CreateProfile` copies activity and reward lists; later-added nested profile fields need an explicit copy/migration decision. Do not assume its one-level `DeepCopy` recursively handles arbitrarily nested future schemas. Settings UI callbacks and scanner consumers must agree on the shape.

Validation without a game client

The inspected repository has no conventional npm or Python application test command. A Lua syntax check, if an appropriate Lua compiler is already available, can catch parse errors but cannot supply Blizzard frames, events or restricted API semantics. Use source review and a small future stubbed test harness for pure routing/timing calculations, then actual client acceptance for integration. Do not present an absent test suite as passing.

A practical future harness can supply a minimal `RS` table, load the independent module being tested, and replace time/map APIs with controlled functions. Keep such tests separate from the live client and record precisely which APIs were stubbed. Source errors that occur only in combat or after asynchronous map loading still need the real boundary checked.

Normal close and data retention

Use `/rs stop` to end a navigation chain and clear the active pin; hiding the route window is a different action. Finish the game session through normal logout or quit so the client can write SavedVariables. Back up saved data with the game closed before changing versions. Do not use `/rs reset` or `/rs resettime` as generic troubleshooting commands: they deliberately remove completion or learned timing data.

Use routes profiles and navigation

Build a route from current information

After login the bootstrap detects active expansions, initializes travel tools, stops a stale navigation chain and schedules a delayed scan. Open the appropriate world-map zones if quest information has not populated yet, then use `/rs scan`. The map callback is debounced because map changes can arrive rapidly. A temporarily incomplete scan is not automatically a math failure in the router.

Open the main window with `/rs` or the minimap control. Inspect activity names, zones, travel legs, estimated activity durations and cumulative time. Red urgency and travel color are presentation aids; they depend on model assumptions and current data. Verify that the character can actually do the listed activity before investing in a long trip. Level-based expansion activation can also explain why a route is empty for one character but not another.

Select preferences deliberately

Use the settings/profile interface to create an editable custom profile rather than expecting the default profile to change. Activity order controls inclusion and priority, reward order controls scoring, and zone preferences influence travel choices. Prefer/avoid is a bias, not a hard prohibition. A First zone is handled by partitioning activities and routing its group before the remaining activities. In this source the First-zone branch concatenates stops without retaining the standard route total fields, and reads

`last.mapID/x/y` although ordinary stops wrap coordinates under `last.activity`. That is a concrete source-level concern to reproduce before trusting this preference mode.

The flight model can be toggled with `/rs skyriding`; the source also attempts live aura/mount detection. Inspect which assumption changed when an estimate looks wrong. A ground-mounted or unmounted state can fall back to saved preference. Hearth/teleport availability is read through game APIs and saved ownership hints; an unknown API can trigger a fallback, so a suggested shortcut should still be checked by the player.

Navigate and understand completion

`/rs nav` or `/rs start` begins the stop chain; `/rs stop` cancels it. Selecting a row can jump to a stop, while marking done removes or advances a candidate through the module's supported path. A quest stop with a quest ID does not auto-complete merely because the player stands near its coordinates. The waypoint code waits for appropriate quest events. Non-quest stops can advance after proximity dwell, and portal stops can advance after arriving at a portal or detecting the destination zone.

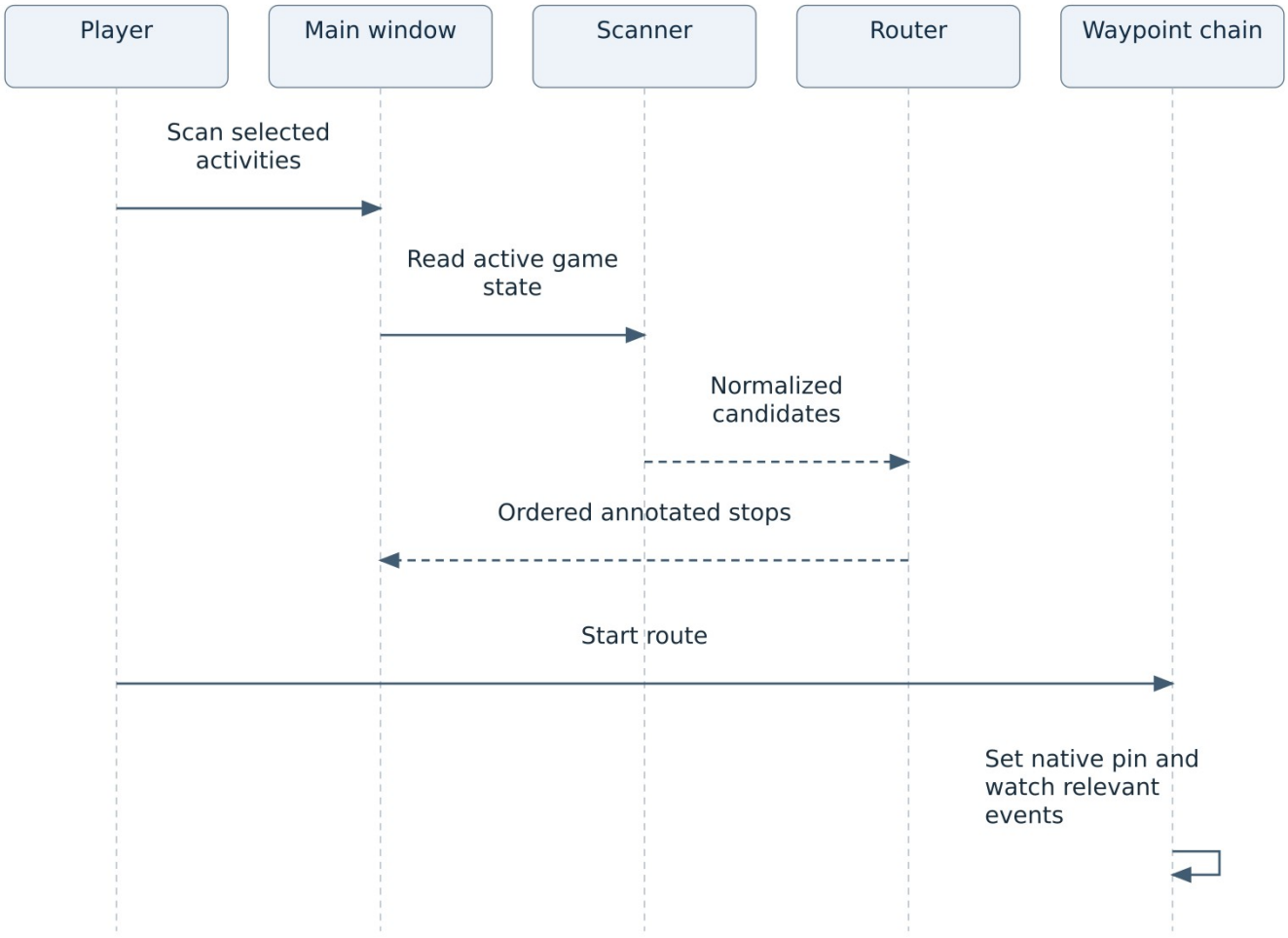


Figure 3 part 1 of 2 Navigate and understand completion



Figure 3 part 2 of 2 Navigate and understand completion

A visited location and a completed task are different facts. If a non-quest activity is still incomplete after a proximity-based advance, inspect its data classification and use the intended manual controls rather than silently trusting the navigation percentage. A completed week can require a turn-in quest rather than an accepted quest ID. The source uses dedicated turn-in IDs for some static tasks; preserve that semantic difference when adding IDs.

Learn from personal timing

Completion/advancement can record a duration. The next route estimate can then use the character's own accumulated average. A single long AFK period should be rejected by the ninety-minute cap, but ordinary slow runs within that range still influence the average. Tooltips can show average, count and last sample. Review those values before blaming the flight model for a long total: travel and activity time are separate components.

`/rs resettime` clears all learned timing through the public slash handler. Although an internal `ResetOne` helper exists, the current handler does not expose arbitrary raw-key selection to the user. Do not document an internal function comment as a supported command. `/rs reset` clears manual weekly completion separately. Both are destructive to a specific data category, and neither is required just to close the window.

Capture a useful problem report

Record source/client version, character level, active profile, selected flight mode, affected activity ID and the small observed symptom. A screenshot of a wrong route helps only when the expected origin, destination and relevant settings are known. `/rs dump` creates a wider character diagnostic snapshot in SavedVariables; examine and redact it locally before sharing any excerpt. It can contain quest, currency, faction, inventory and character details that do not belong in a public issue or handbook.

A good acceptance pass checks login, supported/unsupported level activation, map-data delay, a scan, profile change, a same-zone route, a portal transition, a quest turn-in, navigation stop, timing retention and normal logout. Repeat relevant checks after a client patch; a previously supported API name is not proof of unchanged current semantics.

Continue developing the add on

Add an activity without bypassing normalization

Start in the relevant `Expansions` module. Add a stable ID, display name, map ID, normalized coordinates, activity type, duration/reward data and an appropriate completion source. The map coordinates are fractions between zero and one, not percentages between zero and one hundred. Verify both the map identity and coordinates in the intended client. Adding an entry to a nonexistent older `Core/Zones.lua` path, as an old README passage suggests, will not update the currently loaded expansion data.

Trace the record through expansion aggregation, `GetActiveActivities`, `ScoreActivity`, `BuildRoute`, UI rendering and waypoint completion. Decide whether its activity type belongs in the profile-managed set or is a specialized leveling type. Add a fixture proving it is included when enabled, omitted when disabled/completed and not duplicated by overlapping API sources. For quest-based activities, test accepted versus completed versus turned-in states separately.

Add an expansion module

Use the existing Midnight and Leveling module boundaries rather than copying the entire add-on. Supply the registry data, level range, zone definitions, connectivity, portals, database and activity producer, and add its TOC entries before shared consumers. Check `DetectActive` for overlap and deterministic primary-database choice. A valid registry record is only the beginning: merged zone IDs, hub assumptions, travel tools and profile classification must remain consistent.

Test an eligible and an ineligible level, no active expansion, two active modules, unknown zone width fallback, absent portal connectivity and delayed quest-line data. The Leveling implementation makes asynchronous requests for quest-line information. A request and a read in the same frame cannot be assumed to contain server-returned data. Design retries/events or prefetching explicitly and avoid loops that continually rescan every frame.

Change route ranking with measurable fixtures

Create small synthetic activities in two or three zones and a controlled travel-cost function. Verify each candidate appears once, the first origin is correct, preference affects routing cost but not displayed physical time, urgent stops receive the intended bias, and the final annotated total equals the sum of legs and durations. Include a case with four or more stops so 2-opt actually runs, and a directed-portal case where reversing a segment changes internal edge directions.

Compare final route quality against a simple baseline before claiming an optimization. The heuristic is tuned for a modest candidate set. More candidates mean a pairwise matrix and repeated optimization passes can cost more client time. Measure scan time separately from routing and rendering; a slow map API is not evidence that the optimizer itself is slow.

Evolve settings and saved data safely

The bootstrap fills missing defaults and migrates older timing records. A new field needs a default, a reader, a UI control if appropriate and a migration decision for existing profiles. Test an absent top-level SavedVariable, an old character record, a missing custom profile, an empty ordered list and a partially populated record. Preserve learned timing across weekly reset. Keep completion reset and timing reset distinct in both code and labels.

When adding a destructive control, name what it erases and require the intended user action. Do not add forced reload/checkpoint buttons as routine troubleshooting. The established usage requirement favors passive local diagnostics and preservation of current sessions. A targeted Lua reload during an isolated developer test is different from a product flow that repeatedly demands reloads to operate normally.

Development evidence and publication

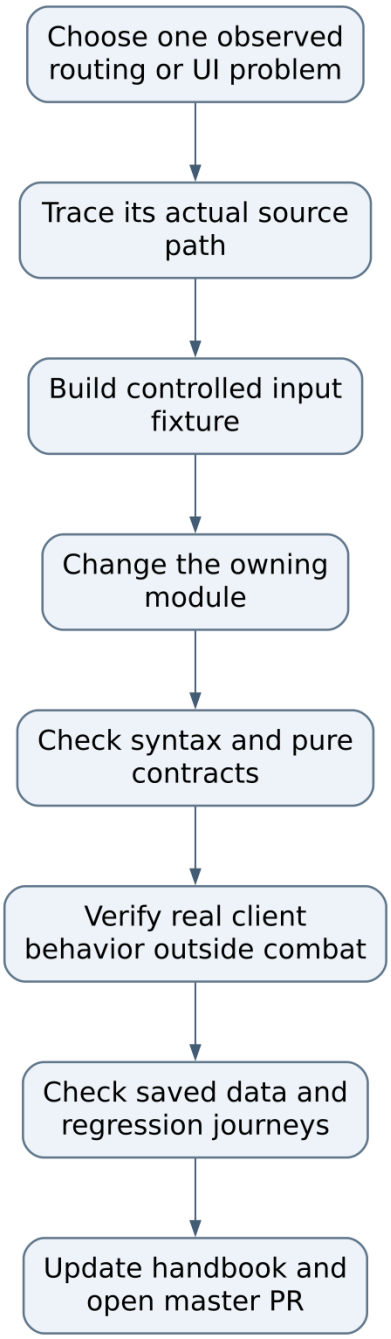


Figure 4 part 1 of 2 Development evidence and publication

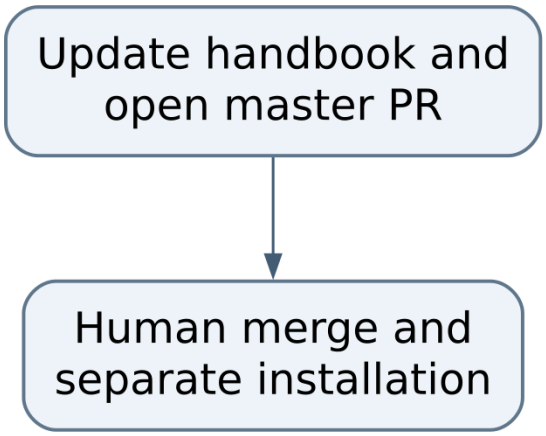


Figure 4 part 2 of 2 Development evidence and publication

There is no implemented repository test framework that automatically proves all these conditions. Keep proposed checks labeled as checks to add until they exist and have been run. Preserve independent ownership of the source repository and the installed game copy. Do not silently synchronize or merge the other RouteSweet repository as part of a small feature, and do not overwrite saved variables or private diagnostic captures during a source update.

Troubleshooting known gaps and recovery

Troubleshoot by pipeline stage

Add-on absent from menu

First evidence. Installed folder and direct TOC location

Safe repair direction. Correct the one selected add-on tree; avoid duplicate nested folders

Error before window opens

First evidence. First Lua error and TOC order

Safe repair direction. Confirm namespace/registry/providers load before consumers

Empty route after login

First evidence. Character level, active modules and map data

Safe repair direction. Wait for the real API response, view target map and scan again

Story quest shown as world quest

First evidence. API source and normalized record type

Safe repair direction. Review scanner classification and duplicate filtering

Completed weekly returns

First evidence. Turn-in ID, reset epoch and manual marker

Safe repair direction. Distinguish accepted/completed/turned-in data; do not clear all settings

Route order differs from shortest distance

First evidence. Score, urgency, zone preferences and portal costs

Safe repair direction. Inspect biased ordering separately from displayed time

Wrong estimate after First group

First evidence. Stop wrapper and totals

Safe repair direction. Inspect group chaining and cumulative offsets for this exact repository

Quest stop will not advance on arrival

First evidence. Quest ID and turn-in event

Safe repair direction. Proximity alone intentionally does not finish a quest

Personal average seems wrong

First evidence. Count, total, last sample and activity key

Safe repair direction. Diagnose sample origin before a targeted data-reset decision

UI disappears in combat

First evidence. combatHide and restore state

Safe repair direction. Check configured visibility behavior before rebuilding frames

Dump seems huge or private

First evidence. Captured sections and character identity

Safe repair direction. Keep raw capture local and share only the necessary sanitized finding

Repository specific findings

The First-zone branch needs a regression fixture for unwrapped final coordinates and complete totals. The shared scanner currently merges quest-log map entries into the world-quest path without the development tree's source-specific story-quest exclusion. Leveling quest-line requests can also need explicit asynchronous prefetch/retry evidence. These are observed source differences, not a claim that an installed character has reproduced each defect. Fix them in this repository with its own tests and review; do not silently treat a neighboring checkout as this revision.

The two version declarations disagree and the README describes several older paths and roadmap items. Reconcile them in a documentation/release task tied to actual source. A developer should not need to infer whether a settings panel exists or whether an entry belongs in the expansion directory. Completion means the TOC, bootstrap display, installation instructions and screenshots of the current interface identify the same release.

The source is rich in client integration but lacks an inspected conventional automated suite. Prioritize small pure fixtures for route membership, directed cost, timing aggregation, weekly reset and saved-schema migration. Add integration observations for delayed map data, restricted values, combat transitions and actual quest completion. Do not use `pcall` alone as proof of correctness: suppressing an exception can still leave a plausible but wrong fallback result.

Data and estimate limitations

Some event rotations and spawn availability are estimated from time or static data. API comments and old quest-ID confirmations are not perpetual guarantees. Record which observation confirmed an ID and which client revision it applied to. Keep unknown IDs explicitly unknown, with a manual completion path where needed. Do not add fabricated IDs to make a table look complete.

The personal average accumulates samples without a fixed moving-window cap. That is simple and stable, but a major objective change can leave old timings influencing new estimates. A future versioned activity key or reviewed migration may be useful. Acceptance should prove old good data is preserved when compatible and invalidated only for a stated reason when the activity's meaning changes.

Routing has preference biases, heuristic limits and directed travel effects. A proposed speed improvement must preserve membership and annotation correctness before optimizing execution time. Use final route cost and useful stop ordering as outcomes; do not claim a better route solely because an intermediate matrix calculation runs faster.

Recovery and safe rollback

Close the game before restoring a previous add-on tree or SavedVariables backup. Restore only the intended add-on and its matching schema-compatible saved data, retaining the failed version and a sanitized error report for diagnosis. Do not replace the whole game profile or another add-on's settings. A source rollback does not automatically reverse a saved-data migration, which is why versioned migration tests and pre-update copies matter.

Stopping navigation with `/rs stop` is ordinary cleanup. Clearing completion or learned timing is a separate data operation. Normal logout/quit gives the client a chance to persist current variables. The actual game installation and NixOS launcher remain external to this source tree and need their own preserved profiles and qualification evidence.

Prioritized continuation plan

First establish reproducible pure test fixtures and source/installed-version identification. Next close the repository-specific scanner/grouping issues with isolated expected-route examples. Then verify saved-data migration and a real supported client journey, including a fresh map-data case and a portal route. Finally improve measured estimate quality and scalability only after the basic contracts are trustworthy. Each repair should name its files, expected user result and retained data; a later developer should be able to prove it without an assistant explaining an earlier session.

Source reference

This handbook describes committed source in COchs-Personal/RouteSweet at `b5ef1dc889ba471878ddf22db3683d33614b1858`. Paths below are relative to this repository. Source-defined behavior is distinguished from a fresh runtime verification; no application or game was launched to author these chapters. The existing quick operating guide remains available separately.

Reading the evidence

A source file explains what the program is designed to do. A test states an expectation and may exercise only one layer. A README describes an intended workflow and can lag implementation. When these disagree, inspect the exact revision and the executable code, reproduce the disagreement in a safe local test, and correct the relevant documentation with the change. Do not treat a passing unit test as proof of rendering, sound, external connectivity, deployment, or save recovery.

File and symbol index

File	Responsibility	Entry points and locations
README.md	Intended user workflow and historical limitations	L1 to L189
RouteSweet.toc	Manifest load order and SavedVariables declarations	L1 to L37
RouteSweet.lua	Defaults, profiles, lifecycle events, reset migration and slash commands	L88 `function RS.DeepCopy(orig)`; L103 `function RS.DeepCopyList(list)`; L112 `function RS:GetActiveProfile()`; L122 `function RS:CreateProfile(name)`; L133 `function RS:DeleteProfile(name)`; L142 `function RS:GetProfileNames()`; L259 `function RS:DetectArcantinaKey()`
Core/Expansion.lua	Registration, level activation and merged zone/database facade	L35 `function E:Register(name, data)`; L56 `function E:DetectActive()`; L91 `function E:HasActive()`; L96 `function E:GetActive()`; L101 `function E:GetExpansion(name)`; L106 `function E:ListSupported()`; L122 `function E:GetZoneYardWidth(mapID)`
Core/Database.lua	Flight mode detection shared across expansions	L34 `function RS.DB:DetectFlightMode()`
Core/QuestScanner.lua	Candidate collection, filters, scores and manual completion	L12 `local function rewardWeightForPosition(pos)`; L17 `local function buildRewardWeights()`; L36 `local function buildEnabledActivityTypes()`; L48 `local function activityPositionBonus(activityType)`; L71 `local function secsUntilReset()`; L82 `function RS.Scanner:GetActiveActivities()`; L135 `function RS.Scanner:ScanWorldQuests()`

File	Responsibility	Entry points and locations
Core/Routing.lua	Biased cost matrix, nearest-neighbor,2-opt, portal annotation and grouping	L17 function RS.Router:BuildCostMatrix(activities); L79 function RS.Router:NearestNeighbour(activities, costMatrix, startIndex); L136 function RS.Router:TwoOpt(tour, costMatrix); L187 function RS.Router:BatchPortalZones(tour, activities); L232 function RS.Router:BuildRoute(activities, playerMapID, playerX, playerY); L465 function RS:BuildRoute()
Core/FlightTime.lua	Distances, speed/overhead assumptions, portals and shortcuts	L51 local function getHubMapID(); L65 local function hearthMapID(); L73 local function hearthAvailable(); L98 local function arcantinaOwned(); L111 local function arcantinaAvailable(); L120 function RS.Flight:ArcantinaCDRemaining(); L129 local function arcantinaCost(fromMapID)
Core/TravelTools.lua	Spell/tool registry, cooldown and availability helpers	L17 function TT:GetSpellCDRemaining(spellID); L39 function TT:IsSpellReady(spellID, bufferSecs); L47 function TT:IsSpellKnown(spellID); L155 function TT:Init(); L184 function TT:Refresh(); L240 function TT:GetAvailable(); L247 function TT:GetBestToolTo(destMapID)
Core/Timing.lua	Per-character sample schema, average selection, clamps and resets	L51 local function ensureTable(); L61 local function storageKey(activity); L71 local function getRecord(activity); L78 local function putRecord(activity, elapsed); L96 function T:GetDuration(activity); L108 function T:GetPersonalAvg(activity); L115 function T:GetStats(activity)
Core/Waypoint.lua	Native pins, stop chain, completion events and ticker cleanup	L36 local function clearNativePin(); L44 local function setNativePin(activity); L73 local function distanceToStop(activity); L87 local function announce(index, activity); L99 local function highlightActiveRow(index); L105 local function advanceTo(index); L136 local function onProximityTick()
Core/DataDump.lua	Private character diagnostic collection	L242 local function safeGet(fn, ...); L247 local function buildQuestLogLookup(); L263 local function questStatus(qid, log); L277 local function buildFactionNameIndex(); L290 local function factionData(key, fid, nameIndex); L324 local function currencyData(key, cid); L333 local function bagItemCount(key, itemID)

File	Responsibility	Entry points and locations
UI/MainFrame.lua	Route rows, interaction callbacks, toolbar and progress	L92 `function RS.UI:Init()`; L588 `function RS.UI:UpdateSkyBtn()`; L659 `function RS.UI:GetOrCreateRow(index)`; L963 `function RS.UI:HighlightActiveRow(index)`; L978 `function RS.UI:SetRowColor(row, isOdd)`; L989 `function RS.UI:Refresh()`; L1158 `function RS.UI:SetWaypoint(activity, rowIndex)`
UI/SettingsFrame.lua	Profiles, category ordering, zone preferences and visibility settings	L64 `local function setBG(frame, r, g, b, a)`; L71 `local function getRelevantRewards(activityOrder)`; L117 `local function getEditableProfile()`; L128 `local function makeScrollFrame(parent, w, h)`; L147 `local function makeSectionHeader(parent, label, startOpen, onToggle)`; L185 `local function findRealIndex(tgt, entryId)`; L192 `local function buildWeightRows(settings, listKey, cont, rowsTable, list, isLocked, yOffset)`
UI/MinimapButton.lua	Toggle, scan and position interaction	L17 `function RS.Minimap:Init()`; L126 `function RS.Minimap:UpdatePosition()`; L140 `function RS.Minimap:SetShown(shown)`
Expansions/Midnight/Zones.lua	Static activity definitions and map connectivity	L1 to L284
Expansions/Midnight/Database.lua	Expansion-specific quests, currency, factions and unlock state	L204 `function DB:GetPreyProgress(difficulty)`; L219 `function DB:IsPreyCapped(difficulty)`; L226 `function DB:GetPreyRenownLevel()`; L236 `function DB:GetPreyFeatures()`; L295 `function DB:ScanPreyWidget()`; L319 `function DB:GetActivePreyQuest()`; L343 `function DB:GetActivePreyZone()`
Expansions/Midnight/Activities.lua	Expansion dynamic activity integration	L18 `local function buildDungeonActivity()`; L53 `local function buildSoireeActivities()`; L154 `local function buildHousingActivity()`
Expansions/Leveling/Zones.lua	Leveling zone registration and portals	L1 to L150
Expansions/Leveling/Activities.lua	Quest-line and bonus-objective discovery	L41 `local function requestQuestLines(mapID)`; L53 `local function scanQuestLog(mapID, seen, results)`; L130 `local function scanQuestLines(mapID, seen, results)`; L189 `local function scanBonusObjectives(mapID, seen, results)`

File	Responsibility	Entry points and locations
Expansions/Leveling/Database.lua	XP scoring and campaign/quest classification	L20 `function DB:GetWarbandXPBonus()`; L101 `function DB:GetQuestXP(questID)`; L115 `function DB:ScoreQuest(questID, questType, estimatedDuration)`; L149 `function DB:ClassifyQuest(questID, questLineInfo)`; L189 `function DB:IsDelversCall(questID)`; L198 `function DB:GetDelversCallStatus()`; L233 `function DB:ScoreDelversCall(questID, isComplete)`

Keeping the handbook useful

When code changes, update the chapter that explains its contract, the relevant diagram, the change recipe, and this pinned source register. Keep the Word and PDF editions synchronized with the Markdown chapters. Record what was actually tested, what remained unverified, and which saved data must survive an upgrade. A future maintainer should be able to trace an instruction to a file or a test without relying on a previous conversation.